

CMR SPECIALIST · v2.0

Full User & Operator Manual

Every capability of the system, and a complete guide to the operator console — what each card does, what each button triggers, and how work flows from complaint to verified resolution.

Item	Detail
System	Lean case-centered agent (spec v2.0) + sealed tool module (CHECK/WRITE over mock CMR DB)
Console	http://localhost:8080 - dark two-column screen, API key only
API docs	http://localhost:8080/docs — interactive docs for every endpoint
Health	http://localhost:8080/health — readiness, database, queue age, worker heartbeat

DESIGN RULE. You need two processes running: the web server (console + API) and the worker (executes queued tasks). The console never executes work itself — it only submits, approves, and displays.

1 · What the system can do

Six capability groups. Everything below is live and tested (20 automated checks).

Capability	What it means in practice
1 · Cases	Open a case from free text; every message preserved verbatim; full state (messages, fields + revision counts, tasks, outputs, recent events) on one screen.
2 · Fact extraction	NIN, phone, email, plate/chassis, Remita RRR, payment date, account, names — each with confidence, source message, and extraction run. Corrections create new versions; originals are never overwritten.
3 · Knowledge Q&A	Upload procedure documents (txt/md/pdf) → auto-chunked versions → publish to activate. Questions answered only from active versions with citations; no evidence means a clear fallback, never a guess.
4 · Task automation	Queue payment_reconciliation, change_of_ownership, or general_support tasks. Missing facts park the task with per-field prompts; follow-ups resume it automatically.
5 · Approvals	Sensitive steps pause in waiting_approval. Approve or reject with a reason from the console; rejections record why and fail the task visibly.
6 · Tool module (CHECK/WRITE)	8 read-only CHECK formulations (profile, NIN, vehicle, owner, buyer, receipt, certificate, portal health) and 7 WRITE formulations (token resend, transfer, payment confirm/link, certificate resend/renew/correct) fired only by the sealed module against the mock CMR DB — every firing validated, authorized, idempotent, and audit-logged.

What it deliberately cannot do: touch the real CMR database (mock only, behind a conformance gate), send real notifications, read scanned images (no OCR yet), or claim an external event occurred without a verified tool result.

2 · Getting started

Start here

```
cd /Users/mac/Desktop/cmr_specialist
cp .env.example .env # set real HARNESS_API_KEY + TOOL_MODULE_TOKEN
pip install -r requirements-dev.txt

# Terminal 1 – web (console + API)
python -m uvicorn harness.api.main:app --host 0.0.0.0 --port 8080

# Terminal 2 – worker (executes queued tasks)
python -m harness.worker.worker

curl -s localhost:8080/health # expect status ok, database connected
```

First five minutes in the console

- Paste your **API key** in the header box (default **cmr-secret-key-2026**) and press **Health** — the activity log should print the health JSON.
- In card **1**, type *How do I renew my CMR certificate?* and press **Submit work**. The case id fills in automatically; badges show the intent and extracted fields.
- In card **4**, upload a procedure text file, then press **Publish** for the returned version id. Ask the question again — the answer now carries citations.
- In card **3**, queue a **payment_reconciliation** task. If facts are missing it parks with prompts — answer them in card **2** and watch it resume and complete.
- Press **Refresh task** any time to poll; the worker picks up queued work within ~2 seconds.

3 · Operator console — screen guide

Layout: dark two-column screen. **Left column** = actions (cards 1–4). **Right column** = state and control (case state, approval, activity log). The header holds the title, the API key box, and the Health button. On narrow screens the columns stack.

Header

Element	Does
Title (CMR Specialist...)	Identifies the console and version.
API key box	Sent as X-API-Key on every mutating request. Must match the server's HARNESS_API_KEY.
Health button	GETs /health and prints status, version, database, queue age to the activity log.

Card 1 · New case (question or task)

Element	Does
Text box	The complaint or question. Example knowledge: renewal steps. Example task: paid RRR text.
Case title (optional)	Human label; defaults to the first 80 characters of the text.
Submit work	POST /cases. Creates the case, classifies intent, extracts facts, and — for tasks — queues the first task. Auto-fills the case-id box and the task-id box if a task was created.
Reset	Clears the local boxes and output pane only. Nothing is deleted server-side.

Card 2 · Follow-up on case

Element	Does
Case id box	Which case the follow-up attaches to (auto-filled after Submit).
Follow-up text box	Extra facts or a new question on the same case.

Element	Does
Attach to case	POST /cases/{id}/messages. Re-runs extraction (new revisions, never overwrites) and resumes any waiting_for_input task whose missing facts just arrived.

Card 3 · Queue automation task

Element	Does
Task type dropdown	payment_reconciliation · change_of_ownership · general_support.
Approval dropdown	never (auto-run) · sensitive (pause on sensitive steps) · always (pause before any write).
Instructions box	What to do; falls back to the card-1 text if left empty.
Queue task	POST /cases/{id}/tasks. Returns the task id into the task-id box; use an idempotency key via API for safe retries.
Run worker once	Informational only — the real worker runs server-side. Then poll with Refresh task.

Card 4 · Knowledge base

Element	Does
File picker + Upload	POST /knowledge/documents (txt/md/pdf). Returns document + version ids and chunk count; the version id auto-fills the publish box.
Version id + Publish	POST /knowledge/versions/{id}/publish. Activates the version for retrieval (archives siblings). Only active versions can ground answers.

Case state card (right column)

Element	Does
Badges line	Live summary: case short-id, each extracted field with revision (phone v2), each task with status (payment_reconciliation: waiting_for_input).
Output pane	Full JSON of GET /cases/{id} or GET /tasks/{id}: messages, fields, plans, steps, approvals, results, error refs.
Refresh case / Refresh task	Re-fetch and repaint. Poll Refresh task while a worker runs.
Task id box	Which task Refresh task and the Approval card act on (auto-filled on queue).

Approval card

Reason box (optional, required for meaningful rejects) plus **Approve / Reject** buttons → POST /tasks/{id}/approvals. Approve returns the task to queued for the worker; reject fails it with your reason recorded. The result is printed to the activity log and the task view refreshed.

Activity log

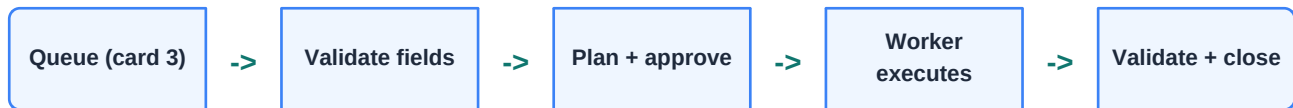
Append-only feed, newest first, timestamped: every action with its HTTP status (create-case 200, task 200 queued, approval 200 ..., task refresh 200). Your first stop when anything looks wrong — it tells you exactly which call did what.

4 · Workflows & task states

Knowledge question



Automation task



Task states you will see in badges and the output pane:

State	Meaning	Your action
queued	Waiting for the worker (picked up within ~2s).	Wait / Refresh task.
leased / running	Worker owns it and is executing steps.	Wait.
waiting_for_input	Missing facts; prompts list exactly what is needed.	Answer via card 2 follow-up.
waiting_approval	Sensitive step needs a human.	Approve / Reject in the Approval card.
completed	Done; result + steps + citations stored.	Review output.
failed	Terminal error with error_ref; nothing was falsely claimed.	Read error_ref; fix input; requeue.
dead_letter	Retries exhausted.	Operator/engineering review.

NOTE. Reposting a task with the same idempotency key (API) returns the existing task — safe retries, no duplicate side effects. A worker restart resumes from the last committed step.

5 · Tool module (CHECK / WRITE)

The sealed database-access layer. The console and API never touch data directly: CHECK formulations verify (profile, NIN, vehicle, owner, buyer, receipt, certificate, portal health); WRITE formulations mutate (token resend, transfer, payment confirm/link, certificate resend/renew/correct) — each gated by approval, idempotency key, and the check verdict that justified it. Every firing is audit-logged with masked PII; replays skip completed keys.

In the console you meet the module indirectly: waiting_approval badges, evidence-backed results in the task view, and audit rows behind every step. Full design: [output/CMR_Specialist_Tool_Module_Architecture.pdf](#).

6 · API quick reference

Method + path	Console equivalent
POST /cases {text, title?}	Card 1 → Submit work
POST /cases/{id}/messages {text}	Card 2 → Attach to case
POST /cases/{id}/tasks {task_type, instructions?, approval_required?}	Card 3 → Queue task
GET /cases/{id}	Refresh case
GET /tasks/{id}	Refresh task (steps, approvals, result/error)
POST /tasks/{id}/approvals {decision, reason?}	Approval card buttons
POST /knowledge/documents (file)	Card 4 → Upload
POST /knowledge/versions/{id}/publish	Card 4 → Publish
GET /knowledge/documents	List documents + version statuses
GET /health	Header → Health

Mutating calls need header X-API-Key. Interactive docs with try-it-out: [/docs](#).

7 · Troubleshooting & FAQ

Symptom	Check	Fix
Console actions fail (401/403)	API key box matches server HARNESS_API_KEY?	Correct the key; press Health.
Tasks sit in queued	/health oldest_queued_job_age_s rising	Start/restart the worker process.
waiting_for_input won't clear	Task view → waiting_reason.missing	Post exactly those facts in card 2.
waiting_approval stuck	Task view → approvals list	Approve or Reject with a reason.
Answers never cited	Version published? (card 4)	Publish the version; only active versions ground answers.
PDF upload rejected	.pdf/.txt/.md only; text extractable?	Scanned-image PDFs need OCR (not yet built).
Duplicate tasks	Same idempotency key reused?	That is the protection working — one task, one id.
Port 8080 busy	Isof -ti:8080	Kill the holder, restart the web process.

FAQ

- **Do I need the worker running?** Yes for anything beyond Q&A; — tasks queue without it and execute when it starts. Nothing is lost; leases expire and are reclaimed.
- **Where is my data?** SQLite at data/cmr_cases.db locally (or Postgres via DATABASE_URL). Back it up before upgrades.
- **Can the agent invent an answer?** No — empty retrieval yields the bounded fallback, and external claims require verified tool results.
- **Who approved a write?** The task view lists every approval with approver, decision, reason, and time; the module audit log holds the matching data-side row.